

Grounded Chain-of-Evidence for Spatial Visual Question Answering

Wonmin Kim Hyun Woo Yu Anushka Purohit Tanishq Patil

University of California, San Diego

CSE 252D: Advanced Computer Vision (Spring 2026) – Design Document, Week 5

{wok016, hyy019, apurohit, t5patil}@ucsd.edu

1. System Architecture

The Spatial Evidence Agent (SEA) is a training-free, three-agent pipeline that answers binary spatial questions about RGB images. Rather than trusting a single VLM, SEA chains a language Planner, a vision Executor, and a geometric Critic into a bounded active-perception loop that produces a verified answer or abstains with an explicit failure-mode label.

1.1. Pipeline Overview

The diagram below shows the full control flow. Arrows represent state transitions; the correction loop runs at most $k \in \{1, 2, 3\}$ times before the system verifies the answer or abstains.

```
START -> Planner -> Executor
      -> Critic -> Output (verified)
|-> Correction -> Executor (loop)
|-> Abstain (max k or parse fail)
```

1.2. Orchestration Layer

LangGraph [1] (StateGraph) manages the pipeline. Every node receives and returns a single AgentState object, keeping all inter-agent communication strongly typed and serialisable. The configurable section of each RunnableConfig carries the LLM handle, executor backend config, and critic settings so nodes are stateless functions.

- state.py-AgentState (pipeline), SpatialEvidenceGraph (output), CriticEvidence (per-claim evidence)
- pipeline.py build_graph() compiles the StateGraph; run_pipeline() is the public entry point
- Routing functions (route_after_planner, route_after_critic) select edges based on state flags: error, iteration, and executor/critic agreement

1.3. Key Design Decisions

- Training-free: all four models run off-the-shelf with no fine-tuning.
- Fail-safe abstention: the system refuses to answer rather than guessing when evidence is ambiguous.

- Active perception: the Critic crops around the disputed bounding-box region and re-submits the crop to the Executor for up to k iterations.
- Geometry-only fallback: if the Executor backend is unavailable, the Critic’s geometric verdict alone drives the answer.
- Deterministic mock mode: Patches detector and depth modules for zero-dependency testing.

2. Representations & Proposed DSL

2.1. Spatial Predicate DSL

All spatial relations are mapped to one of eight predicates, forming a closed DSL that the Planner produces. The Critic then applies the output via deterministic geometric rules. The margin $\delta = 0.02$ in the normalized image coordinates is the slack threshold below which centroids are considered equal.

Relation	Geometric Rule	Notes
left_of	$cx(obj1) - cx(obj2) < -0.02$	Horizontal centroid delta
right_of	$cx(obj1) - cx(obj2) > +0.02$	Horizontal centroid delta
above	$cy(obj1) - cy(obj2) < -0.02$	Vertical centroid delta (y down)
below	$cy(obj1) - cy(obj2) > +0.02$	Vertical centroid delta
behind	$depth(obj1) - depth(obj2) > +0.02$	Median depth in bounding box
in_front	$depth(obj1) - depth(obj2) < -0.02$	Median depth in bounding box
on	$cy\ diff < -0.02\ AND\ IoU > 0.05$	Vertical overlap required
contains	$coverage(obj2\ in\ obj1) > 0.70\ AND\ area(obj2) < area(obj1)$	Spatial enclosure

Table 1. Spatial Predicate DSL Rules

2.2. Spatial Evidence Graph (SEG)

The SEG is the structured output of every pipeline run. It is a Pydantic model that combines scene-graph semantics with explicit geometric proofs:

```
SpatialEvidenceGraph {
  question, obj1, obj2, relation # scene-graph triple
  answer: bool | None           # True yes, False no,
  ↳ None=abstain
  answer_str: 'yes' | 'no' | 'abstain'
  confidence: float             # 0.9 verified, 0.7
  ↳ geo-only, 0.0 abstain
```

```

evidence: List[CriticEvidence] # one record per
↪ iteration
iterations: int # 1..k
crop_history: List[CropRegion] # bounding boxes
↪ examined
verified: bool # True only when
↪ Critic confirms claim
failure_mode: 'detector_miss' | 'depth_noise' |
↪ 'vlm_bias' | 'planner_parse_error'
}

```

2.3. Critic-Evidence Rule

Each loop iteration adds one CriticEvidence record containing: the raw bounding boxes from GroundingDINO [3] (normalized [0,1] coordinates), median depth estimates from Depth Anything V2 [4], the computed $dx/dy/dz$ centroid deltas, the IoU, the applied rule string, and a failure_reason string if the verdict is false. This makes every decision fully auditable and serializable to the evidence JSON graph.

3. Agent Roles

Agent	Model / Backend	Input	Output
Planner	GPT-4o-mini	Nat-lang q.	{obj1, obj2, rel}
Executor	LLaVA-1.5 [2]/GPT-4V	Img+parsed pred	yes/no + claims
Critic	Grnd. DINO [3]+Depth [4]	Crop+queries	BBox, depth, verdict

Table 2. Agent Roles and I/O. Correction is a deterministic state-update node, not a learned agent.

3.1. Planner

The Planner converts a free-form English question into a typed triple (obj1, obj2, relation) using a strict system prompt with the eight-relation DSL. It uses LangChain’s with_structured_output when available; otherwise it JSON-decodes the raw response via extract_json_object(). A conservative regex fallback handles common phrasings if both LLM paths fail (e.g., “Is X to the left of Y?”). Strict mode raises on any parse failure.

3.2. Executor

The Executor queries a VLM (LLaVA-1.5 [2] locally, or GPT-4V/GPT-4o via OpenAI-compatible API) with EXECUTOR_SYSTEM, which mandates a JSON record containing an answer, a 0-1 confidence score, and two object claims. The claims carry a visible flag so the Critic knows whether to expect detection success. If the backend fails and strict mode is off, the Executor degrades gracefully by forwarding the Planner nouns to the Critic as claims.

3.3. Critic (Geometric Engine Rule)

The Critic is the core verification component. It: (1) detects obj1 and obj2 using Grounding DINO [3] (open-vocabulary, dot-separated prompt); (2) estimates a normalized depth map with Depth Anything V2 [4]; (3) applies the

DSL rule for the parsed relation; (4) logs a CriticEvidence record. On failure, it schedules a CropRegion around the disputed bounding boxes for the next iteration. The Critic never guesses if detection misses or the relation rule fails, critic_passed = False.

3.4. Correction Loop and Output/Abstain

The correction_node increments the iteration counter and sets current_crop to the region the Critic computed. The executor re-runs on the cropped image. Agreement between executor_answer and critic_passed triggers the output_node, which assembles the final SEG with verified=True. After max k failures, or when the Planner cannot parse the question, the abstain_node is called instead, classifying the failure as one of: detector_miss, depth_noise, vlm_bias, or planner_parse_error.

References

- [1] LangChain, Inc. LangGraph: Building stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph>, 2024. 1
- [2] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. In *NeurIPS*, 2023. 2
- [3] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection, 2023. 2
- [4] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao. Depth anything V2. *arXiv preprint arXiv:2406.09414*, 2024. 2